

AI Fluency Lab Workbook

Hands-On in Claude — For Students

This workbook is your copy. Every lab below is something you **run live in Claude**, not something you read about. Each step has an exact prompt to paste in, and a short note on *what to watch for* — this is where you actually see Delegation, Description, Discernment, and Diligence happen, instead of just hearing them defined.

Before you start:

- Open Claude in a fresh conversation for each lab unless a step tells you otherwise.
 - Where a prompt says `[bracketed text]`, replace it with your own real content — your own code, your own stack, your own past conversation. The exercise only works with real material, not placeholders.
 - Nothing here requires special access — plain Claude.ai or the Claude app is enough.
-

Lab 1 — Audit Your Last AI Interaction

Module 1 • 10 minutes • Individual, Claude-assisted, then pair-share

Objective: Use Claude itself to hold up a mirror to how you already work with AI.

Step 1. Find your most recent real AI conversation — code help, an email draft, anything. Have the prompt you sent and the response you got ready to paste.

Step 2. In a new Claude conversation, paste:

```
I want to audit a past AI interaction using the AI Fluency 4D Framework
(Delegation, Description, Discernment, Diligence). Here's what happened:
```

```
[paste or describe your original prompt, then the AI's response]
```

```
For each of the 4 Ds, tell me:
```

- ```
1. What I did well
2. What I likely missed
3. One specific thing I could have done differently
```

```
Be direct and specific. Don't just validate me — I can handle honest
feedback.
```

**What to watch for:** Claude will usually surface a gap in Discernment ("did you actually verify this before using it?") or Diligence ("did anyone else see this output — did they know AI was involved?") that you hadn't consciously registered. That's the exercise working, not Claude being harsh.

If the audit comes back feeling too agreeable or generic, that itself is a live Discernment lesson: push back —

Be more critical. Assume I'm a professional who wants the honest gaps, not reassurance.

Notice how the answer changes. That shift is exactly what "confident tone ≠ correctness" means in reverse: default helpfulness can also be shallow, and it's on you to ask for the sharper version.

**Debrief:** Pair up. Share the one D that Claude's audit flagged as your weakest. Hold onto it — Lab 5 comes back to it.

---

## Lab 2 — Spot the Hallucination

**Module 2 • 20 minutes • Pairs, live in Claude**

**Objective:** Build the reflex of verifying a specific technical claim before trusting it — regardless of how confidently it's stated.

**Step 1.** Pick a library, framework, or tool your pair doesn't use daily, and ask Claude something specific and detail-heavy about it:

Show me how to configure [connection pooling / retry backoff / a specific feature] in [library name], including the exact parameter names and their default values.

For a harder test, push toward the edges of what's likely to be well-documented:

What are all the configuration options for [an obscure feature] in [library] version [a recent version]? List every parameter with its default.

**Step 2.** Do not take the answer at face value. Open the real source: the official docs, the GitHub source, or `--help` output for that exact library and version. Check every specific claim — parameter names, defaults, behavior.

**Step 3.** Log one sentence, either way:

- *"Claude claimed \_\_\_; the real docs say \_\_\_."* — if you found a discrepancy, or
- *"Verified: no discrepancy found, verification took ~X minutes."* — if it checked out.

**What to watch for:** Claude will often hedge on genuinely obscure details ("I'm not fully certain of the exact default — verify this") — notice that hedge when it shows up, it's useful signal. Also notice the cases where the phrasing is fully confident but still wrong, or where a version-specific detail is stated as if universal. Both patterns are worth naming out loud with your partner.

**Debrief:** How long did verification take, compared to how long a wrong config value would take to surface as a bug in production?

---

## Lab 3 — Prompt Rewrite Sprint

**Module 3 · 40 minutes · Individual, then groups of 3, live in Claude**

**Objective:** Feel the output-quality gap between a vague prompt and a structured one, from the same tool, back to back.

**Step 1.** In a fresh conversation, send your real, natural, first-draft prompt for something on your actual backlog. If you don't have one ready, use this pattern with your own broken code pasted underneath:

Fix this code.

[paste a real function of yours with a real bug]

**Step 2.** Open a **second, brand-new conversation** — this matters, don't let Claude carry context from the first one. Send the same task rewritten with Goal / Context / Constraints / Format:

Goal: This function throws a [real error] on [real line/condition]. Fix it so it [the actual desired behavior] instead of [the actual bad behavior].

Context:

[paste the full function and the call site]

Constraints: Keep the method signature unchanged. Don't introduce new dependencies.

Format: Return a diff, not the full file re-pasted.

**Step 3.** Put both responses side by side. Write down specifically what changed — not just "better," but *what kind* of better: fewer follow-up questions, matched your code's existing style, caught an edge case the vague version didn't.

**What to watch for:** the vague prompt usually makes Claude either ask a clarifying question or silently guess at scope — that guess is Claude filling in gaps *you* left, and it's filling them with a generic assumption, not your actual intent. The structured prompt should need little or no back-and-forth to be usable.

**Step 4 (groups of 3).** Compare your three "weak" prompts. Pick the weakest one as a group and rewrite it together on one screen, narrating out loud why each addition (Context vs. Constraints vs. Format) changes the output.

**Debrief:** Which single addition moved the output furthest for you personally?

---

## Lab 4 — The Flawed PR Review (Claude-Generated, Live)

Module 4 • 30 minutes • Pairs, live in Claude

**Objective:** Practice checklist-driven Discernment — using Claude to generate a fresh flawed sample on the spot, so every pair gets a different one.

**Step 1.** One person in the pair runs this prompt:

```
For a code-review training exercise, write a 30-40 line [pick your stack,
e.g. Node.js / Python / Terraform] function or endpoint that looks correct
at first glance but contains exactly three planted problems: one functional
bug, one security issue, and one comment or docstring referencing a
configuration option or library behavior that does not actually exist.
```

```
Do not tell me what the problems are or where they are. Just give me the
code.
```

**Step 2.** Share that code with your partner. Using the 7-point Discernment checklist (Correctness, Verifiability, Security, Fit with Constraints, Maintainability, Completeness, Reasoning Quality), find all three planted issues in 15 minutes. For the suspicious library/config claim specifically, verify it against real documentation before deciding it's fabricated — don't just guess.

**Step 3.** Once you've made your calls, go back to the same Claude conversation and send:

```
Reveal the three planted issues and exactly where they are in the code.
```

Score yourselves against the reveal.

**What to watch for:** this mirrors real review pressure — you have limited time and have to choose where to look closely. Notice which issue you found first and which you missed entirely; that gap usually maps directly to the checklist dimension you personally skip when you're moving fast.

**Debrief:** Which of the three was hardest to catch, and why?

---

## Lab 5 — Capstone: The Full 4D Loop

**Module 5 • 35 minutes • Individual or pairs, one continuous Claude conversation**

**Objective:** Run one real task through Delegation → Description → Discernment → Refine → Diligence end to end, in a single thread, so you can watch the loop actually close.

**Step 1 — Delegation.** Before opening Claude, write down (on paper or in a notes doc, not in the chat): for a `POST /users/{id}/reset-password` endpoint, what will you delegate to AI vs. design/review yourself, and why. One sentence is enough.

**Step 2 — Description.** In Claude, send:

Goal: Add a `POST /users/{id}/reset-password` endpoint. It must validate the requester's permissions, rate-limit repeated attempts, and return a generic success message regardless of whether the user exists, to prevent user enumeration.

Context: [your real stack, e.g. Node/Express + PostgreSQL — paste an existing route file of yours so Claude matches your style]

Constraints: Follow the error-handling pattern shown above. Flag any new dependency before adding it.

Format: Return the full route handler, plus a one-paragraph explanation of the rate-limiting approach.

**Step 3 — Discernment.** In the same conversation, stress-test the claims instead of trusting them at face value:

Walk through what happens if the same IP hits this endpoint 50 times in a row, and what happens if the user ID doesn't exist. Does this implementation actually prevent enumeration and actually rate-limit, or does it just describe doing so?

Read the answer critically. Check the actual code against what it claims — this is the same "verify, don't just accept" habit from Lab 2, now applied to your own real task.

**Step 4 — Refine the loop.** Based on the specific gap you find, send **one targeted correction** — not a full re-ask:

The rate limiter resets on server restart and isn't shared across instances. Fix that specifically – leave everything else as is.

Notice how much faster this is than starting over, and how it only works because Step 3 gave you something precise to point at.

**Step 5 — Diligence.** Outside Claude, in your own notes, write one line each:

- **Creation:** was this task and this data appropriate to run through this tool, given your org's data policy?
- **Transparency:** how would you disclose AI involvement in the real PR description?
- **Deployment:** what will *you* personally test before this merges, beyond what Claude showed you?

**Debrief (full group):** A few people walk the room through their actual Claude conversation, live, step by step. Then: did this capstone touch the weak D you identified back in Lab 1? What's the one thing you'll do differently on your very next real AI-assisted task?

---

## Facilitator note

Every prompt above is designed to be run as written, with the student's own real material dropped into the brackets — that's what makes the exercise land as *their* discovery rather than a demo they watched. Encourage students to let the Claude conversation run a little wild (follow-up questions, unexpected answers) rather than steering it back to script; the off-script moments are usually where a D gets learned, not where the lab goes wrong.